Music Knowledge Graph (MusicKG)

A Semantic Web Application for Music Data Management

Course: Semantic Web 2025/26

Technologies: Python • Django • GraphDB • SPARQL

Dataset: 30 000 Spotify Songs from Kaggle

[112793] Cláudia Seabra

June 30th, 2026

1. Introduction

1.1. Project Overview

The Music Knowledge Graph (MusicKG) is a comprehensive Semantic Web application designed to transform relational music data into a highly interconnected RDF-based Knowledge Graph. Unlike traditional Relational Database Management Systems (RDBMS) that isolate data into rigid tables, MusicKG aims to manage musical entities (Artists, Albums, Tracks, and Genres) as a dynamic network of semantic nodes. This approach provides schema flexibility and allows for complex pattern matching via SPARQL.

1.2. Objectives

The primary goal of MusicKG is to bridge the gap between raw data and semantic meaning. The specific objectives of this project are:

1. Transform a Kaggle Spotify playlist CSV dataset into a semantically rich RDF knowledge graph.
2. Implement a dual-mode RDF storage system: GraphDB as the primary engine, with an automatic `rdflib` fallback mechanism for extreme fault tolerance.
3. Use SPARQL for Create, Read, Update, and Delete (CRUD) operations, as well as an advanced mathematical and inferential engine.
4. Develop a highly responsive Django SST interface utilizing asynchronous updates for data exploration.

1.3. Architectural Refactoring

In the initial iterations of this project, a React Single Page Application (SPA) was used for the frontend. However, upon critical review of the core requirements of the project, it was concluded that an SPA was misaligned with the course objectives. Semantic Web data should be inherently indexable, linked, and SSR-friendly.

Consequently, the architecture was completely refactored. React was removed, and the system was rebuilt using a pure Django Server-Side Rendering (SSR) approach. This architectural pivot ensures that the application state and the GraphDB triplestore are perfectly synchronous.

2. Data, Sources, and Transformation

2.1. Dataset Description

The foundational data was sourced from a Kaggle Spotify dataset[1] containing metadata and audio features for over 30 000 tracks. The source CSV provided essential identifiers, textual metadata, and critical quantitative audio descriptors.

[1] https://www.kaggle.com/datasets/joebeachcapital/30000-spotify-songs

Dataset Statistics

Metric	Count
Total Tracks	32 833
Unique Artists	10 316
Unique Albums	20 728
Genres	6 (pop, rap, rock, latin, r&b, edm)
Attributes per Track	23

2.2. RDF Transformation Process and Modeling (ETL)

The ETL (Extract, Transform, Load) pipeline is driven by a custom Python engine (*convert_to_rdf.py*). To maximize GraphDB read/write performance and prevent inference engine bottlenecks, a Flat Fact Graph Model was adopted.

Instead of creating intermediate blank nodes for audio features, quantitative metrics were mapped directly to the Track nodes using strictly typed XSD literals (e.g., `xsd:float`, `xsd:integer`).

URI Generation Strategy:

- **Tracks:** Deterministic IDs (`http://musickg.org/track/{id}`) are used to guarantee a 1-to-1 mapping with external systems.
- **Artists & Albums:** URL-safe semantic slugs (`http://musickg.org/artist/{name}`) are employed. This critical design choice allows the Django backend to dynamically generate URIs for newly created entities via UI forms without relying on third-party ID generators.

2.3. Data Storage and Serialization

The transformed graph is serialized into an uncompressed N-Triples (`.nt`) format. This specific serialization was chosen over Turtle/XML because it allows for rapid, memory-efficient HTTP stream ingestion into GraphDB, avoiding socket timeouts during bulk loading.

3. Operations on Data (SPARQL)

The logic layer of MusicKG relies exclusively on SPARQL 1.1, delegating heavy computational tasks to the GraphDB engine rather than the Python server.

3.1. Dual-Mode Storage Architecture

The system implements a Singleton Store pattern (*rdf_store.py*) to ensure extreme resilience:

1. Upon startup, the system executes a `HEAD` request to the GraphDB REST API (port 7200).
2. If the repository is empty, the system initiates a dynamic synchronization of the `.nt` file.
3. If GraphDB is unreachable, the system triggers an automatic and transparent fallback to the `rdflib` library in read-only mode, guaranteeing zero downtime.

3.2. Documented Semantic Inference

The application avoids opaque and automatic RDFS/OWL reasoners in favor of explicit and rule-based SPARQL inferences. This approach prioritizes 'White-Box' inference, where logic is

transparent and easily debuggable via standard SPARQL IDEs. This guarantees predictable performance and allows for complex mathematical reasoning:

1. **Semantic Similarity Inference (Vibe & Related Artists):** The system infers "Related Artists" directly within the GraphDB engine by calculating the intersection of shared `music:inGenre` predicates across nodes. For audio "Vibes", the system uses SPARQL mathematical functions like `ABS(?targetEnergy - ?energy)` to infer and rank tracks that share a similar acoustic profile.

2. **Temporal & Chronological Inference:** In the Timeline analytics, the system mathematically infers the "Decade" of an album purely via SPARQL using the operation: `BIND (FLOOR(?year / 10) * 10 AS ?decade)`. This creates a dynamic categorization layer at runtime without altering the base data.

3.2. Structural Integrity and Validations (ASK)

To maintain graph integrity during user-driven data entry, the system utilizes SPARQL `ASK` queries. Before a new Album, Track, or Artist is inserted, an `ASK` constraint evaluates both direct properties and inferred relationships to prevent the creation of duplicate semantic nodes.

Furthermore, regarding data persistence, maintaining referential integrity while performing updates and deletions proved to be a significant challenge. Initial implementations utilizing `OPTIONAL` clauses within SPARQL `DELETE` queries were found to be complicated, often leaving orphan triples in the knowledge graph. To resolve this, `DELETE` queries were refactored to employ exhaustive `UNION` blocks, to ensure the unconditional removal of all predicates connected to a node before any re-insertion and guarantee data consistency.

3.3. Semantic Exports (DESCRIBE & CONSTRUCT)

To comply with Semantic Web interoperability standards, the system allows users to extract sub-graphs in the Artist page. Using `DESCRIBE <uri>`, users can view raw RDF triples, while `CONSTRUCT` queries allow the downloading of self-contained Turtle (`.ttl`) files for specific entities.

4. Application Functionalities (UI/UX)

The user interface was built using Django *Templates* and *Tailwind CSS*. Figure 1 and Figure 2 represent the initial Home Page and Search Page, respectively.

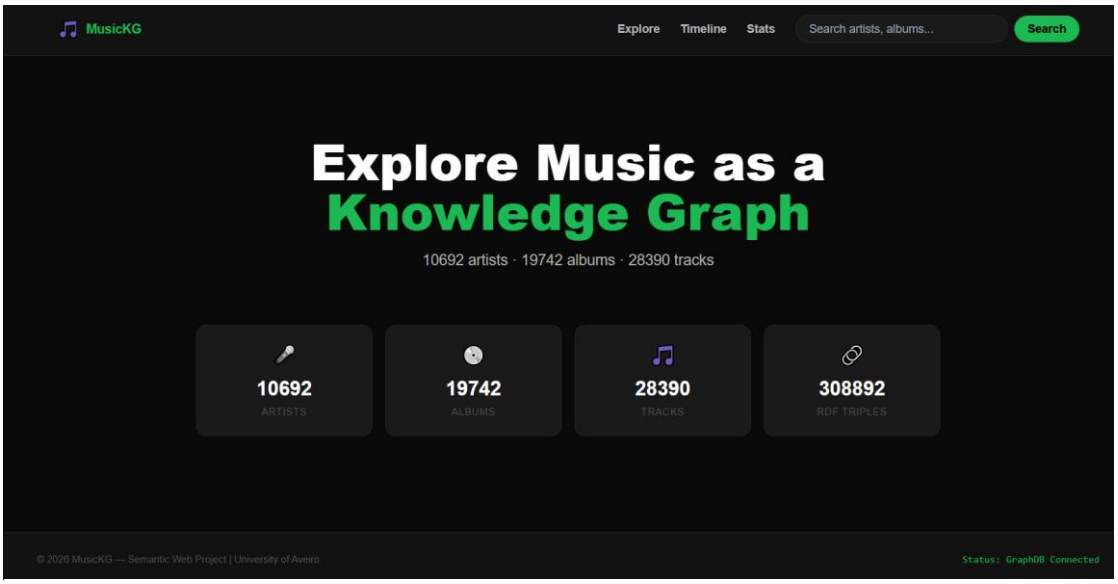

Figure 1: Home Dashboard

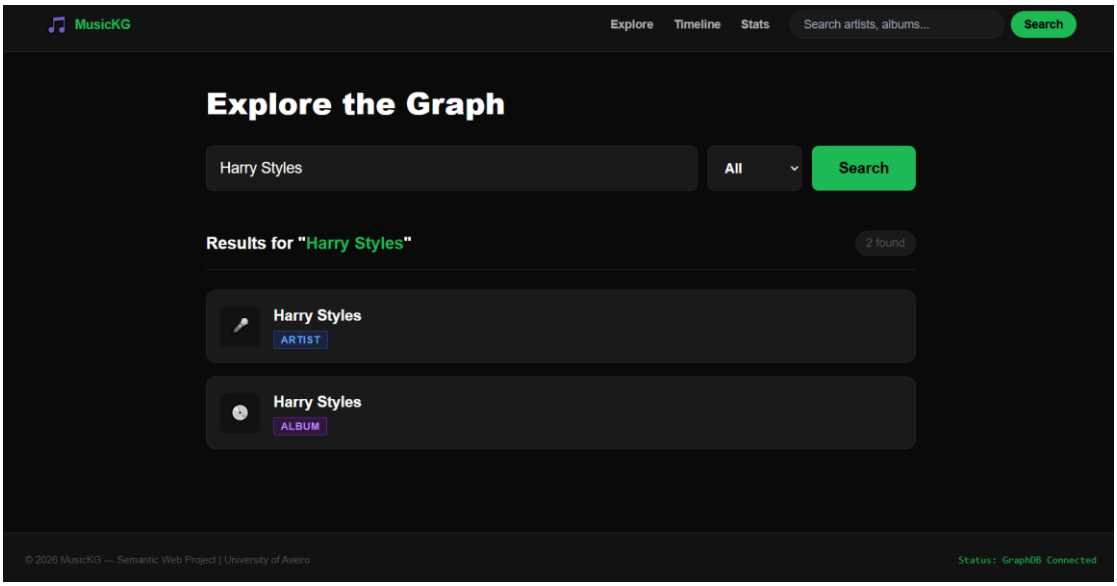

Figure 2: Search Page

4.1. Artist Discovery and Profile Dashboard

The Artist page serves as a central hub for semantic navigation. As shown in Figure 3, the interface features a persistent sticky navigation sidebar for enhanced user experience and dynamically lists the artist's discography.

- The header dynamically generates a DBpedia URI based on the artist's name.
- Furthermore, the dashboard exposes full CRUD capabilities. Users can add new tracks, create empty album nodes, and permanently delete entities directly from the UI.

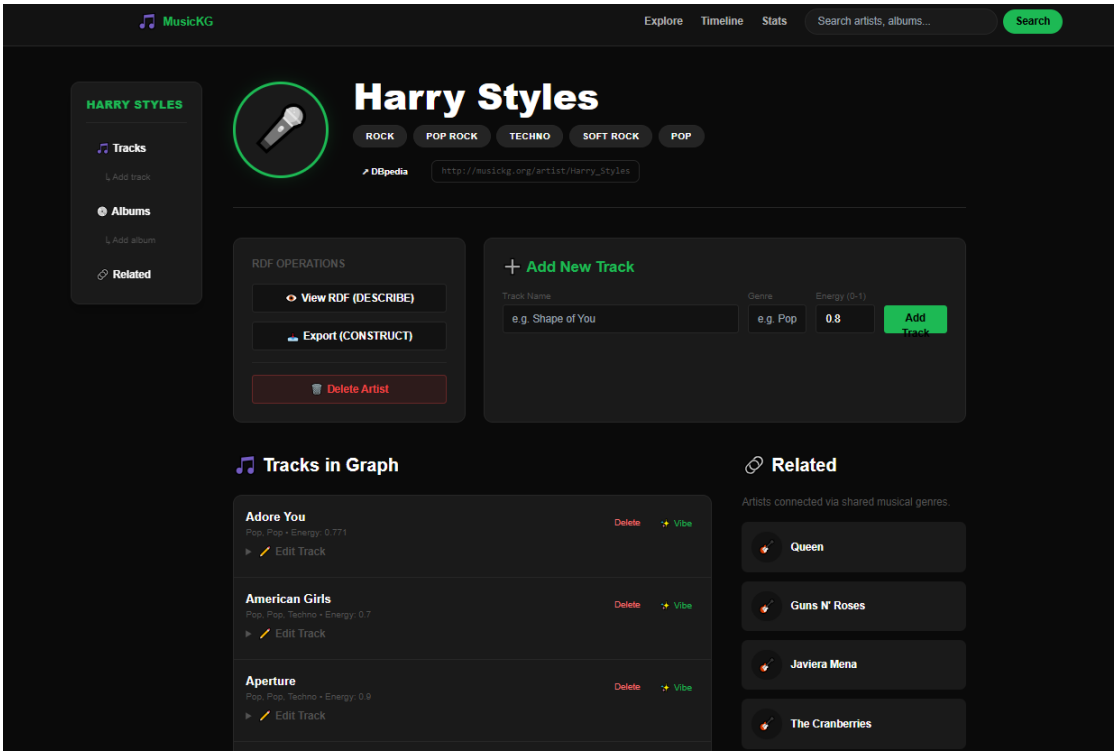

Figure 3: Artist Dashboard

4.2. Advanced Discography Management (Album Detail)

The Album Detail page, illustrated in Figure 4, allows for relationship management. Beyond being able to edit literal values (like Release Year), users can manipulate the Graph edges. The "Unlink" feature executes a SPARQL query that removes the `music:inAlbum` edge, converting the track into a "Single" without deleting the track node itself from the system. Furthermore, the user can also delete a track node or the album entirely.

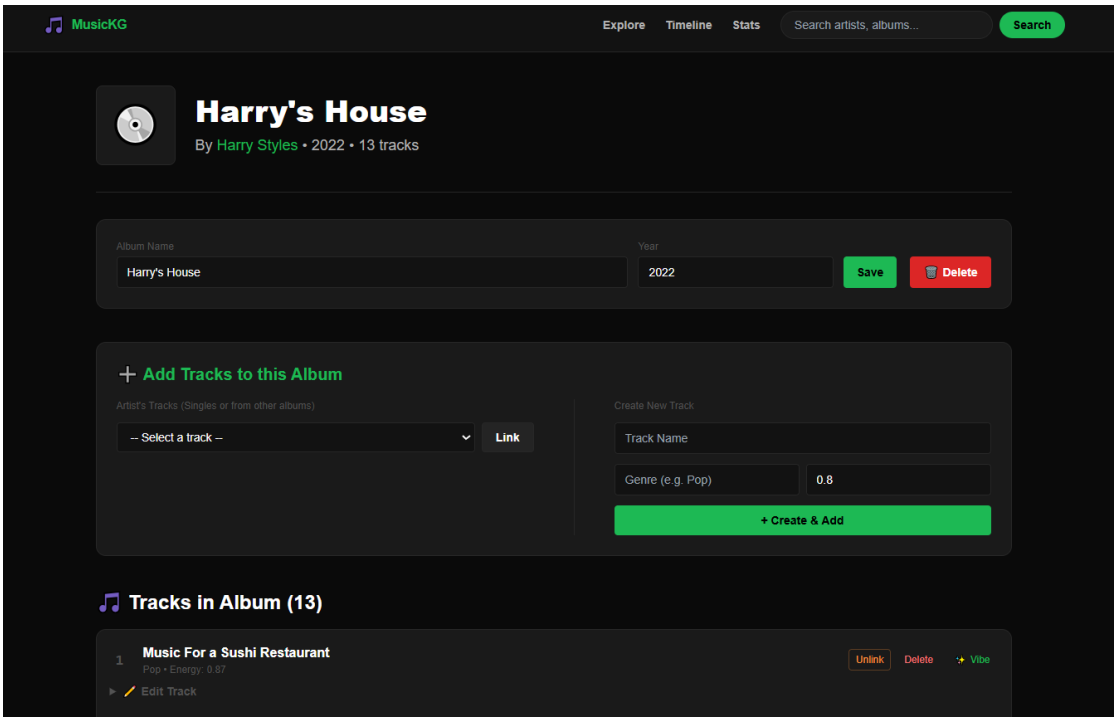

Figure 4: Album Detail Management Page

4.3. Timeline and Asynchronous Exploration

The system provides several pages that provide diverse perspectives on the music knowledge graph.

Timeline

The Timeline view in Figure 5 introduces asynchronous data loading. Users can filter albums by Decade and Alphabetical variables via URL `GET` parameters, which are translated into SPARQL `FILTER(STRSTARTS())` constraints. Clicking "Load More" fetches JSON data from Django and appends it to the DOM without breaking the user's scroll position or page state.

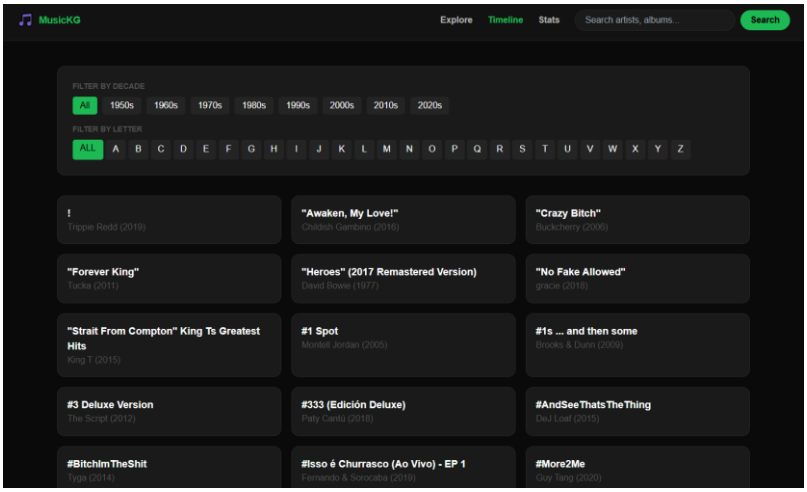

Figure 5: Timeline Interface

Audio Explorer

As shown in Figure 6, the Audio Explorer provides a quantitative analysis interface. It allows users to filter tracks based on numerical audio features (energy levels) and genre constraints. By leveraging `FILTER(?energy >= ... && ?energy <= ...)` in SPARQL, the application provides an interactive discovery layer that allows users to understand and correlate acoustic metrics across tracks.

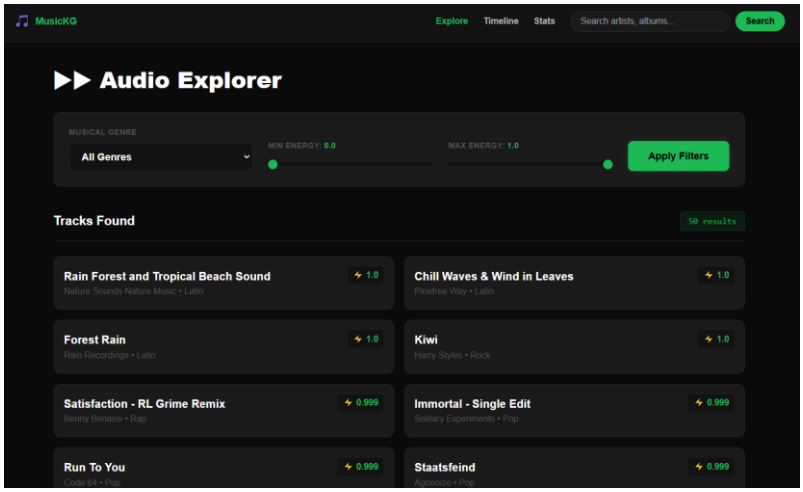

Figure 6: Audio Explorer Interface

Graph Insights

To complement the individual entity exploration, the Graph Insights dashboard (Figure 4) provides an aggregated view of the knowledge base. It uses SPARQL aggregations (`COUNT` and `AVG`) to generate high-level visualizations, such as top-genre distributions and energy trends.

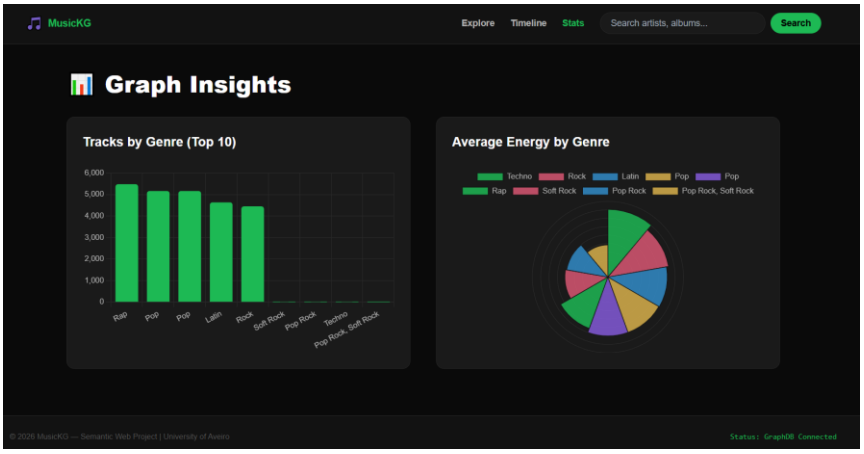

Figure 7: Graph Insights Interface

4.4. System Routes (Django Views)

Category	Endpoint	Method	Functionality
Discovery and Analytics	/	GET	Renders the global homepage and connection status dashboard.
	/search/	GET	Multi-type full-text search across artists, albums, and tracks with dynamic filters.
	/explore/	GET	Interactive exploration based on numerical Audio Feature sliders.
	/timeline/	GET	Chronological and alphabetical matrix with AJAX pagination.
	/stats/	GET	Retrieves aggregated knowledge graph metrics and node/triple counts.
Entities	/artist/<slug>/	GET	Comprehensive semantic profile featuring sticky navigation.
	/album/<slug>/	GET	Album-specific metadata and full track listings.
	/track/<slug>/vibe/	GET	Analyzes acoustic profiles and provides similarity-based track recommendations.
Semantic Exports	/artist/<slug>/raw/	GET	Outputs a text/plain HTTP response generated via SPARQL DESCRIBE queries.
	/artist/<slug>/export/	GET	Triggers a .ttl file download generated via SPARQL CONSTRUCT queries.
Data Ingestion	/create-artist/	POST	Initialization of new Artist nodes with deterministic URI generation.
	/artist/<slug>/create-album/	POST	Creation of new album entities explicitly linked to a parent artist.
	/artist/<slug>/add-track/	POST	Ingestion of new track nodes and automatic association to the artist.
Graph Modification	/album/<slug>/edit/	POST	Atomic updates of album literal properties (e.g., Release Year).
	/track/<slug>/edit/	POST	Atomic updates of track literal properties (e.g., Genre, Energy).
	/album/.../add-existing-track/	POST	Edge creation: Links an existing independent track to an album (music:inAlbum).
Graph Maintenance	/album/.../remove-track/.../	POST	Edge deletion (Unlink): Surgically removes a track from an album, keeping it as a Single.
	/track/<slug>/delete/	POST	Permanent node deletion triggering automated orphan album cleanup.
	/album/<slug>/delete/	POST	Deletes the album node, gracefully converting all its tracks into independent Singles.
	/artist/<slug>/delete/	POST	Cascade deletion of an artist and all their structurally dependent tracks and albums.

4.5. Technical Challenges and System Resilience

Handling Data Heterogeneity

The dataset presented challenges inherent to Linked Data, specifically semantic heterogeneity and inconsistent URI patterns. These issues led to redundant nodes in the graph and routing conflicts within the web application. To address this, a defensive data cleaning layer was

implemented in the backend, utilizing regular expressions and URI normalization to ensure a consistent presentation of entities, despite underlying raw data variations.

GraphDB Persistence and Synchronization Lifecycle
A critical issue identified was the unintended re-injection of initial data upon server restarts. The synchronization logic was initially predicated on a triple-count threshold, which incorrectly triggered a full re-upload whenever deletions caused the triple count to drop. This was mitigated by constraining the synchronization logic to verify whether the repository is strictly empty (zero triples) upon startup. By decoupling the initialization process from the application's runtime state, it was ensured that manual deletions are treated as persistent and authoritative, ending the unintended data regeneration cycle.

5. Conclusions

The development and subsequent refactoring of MusicKG successfully demonstrate the efficacy of Semantic Web technologies in managing complex, highly interconnected musical datasets. By abandoning the SPA architecture in favor of a pure Django SSR application, the project aligns perfectly with web standards, ensuring that data is both human-readable and machine-indexable.

Beyond the successful implementation of a fault-tolerant dual-mode triplestore API, the project highlights the effectiveness of applying complex mathematical inference directly within the SPARQL engine. Crucially, MusicKG is designed as a collaborative knowledge platform. By providing intuitive CRUD operations, the system empowers users to actively contribute to the graph's expansion, allowing them to refine, edit, and contribute their own musical knowledge to grow the repository collectively.

Ultimately, MusicKG serves as a practical implementation of Linked Data principles, proving that structured knowledge graphs provide a superior alternative to rigid relational schemas for domain-specific knowledge exploration.

6. Configuration and Execution

6.1. System Requirements

- Python 3.9+
- GraphDB Free/Standard installed and running locally on port 7200.

6.2. Installation & Setup

Step 1: Install Dependencies and Generate the RDF Graph

```
pip install -r requirements.txt
```

```
python convert_to_rdf.py --csv spotify_songs.csv --data-dir data/
```

This generates the music_kg.nt file required for initialization.

Step 2: Start the Django Semantic Backend

```
python manage.py migrate
```

```
python manage.py runserver
```

Upon startup, Django will automatically detect GraphDB on port 7200. If the repository is empty, it will auto-stream the .nt file into the triplestore.

6.3. Verification

Access the application by navigating to `http://127.0.0.1:8000/` in any browser. On the global Statistics Dashboard (Home page), the system will display the real-time connection status (indicating whether it is operating in "GraphDB" mode or the safe "rdflib fallback" mode).